

Lecture-6

Packet Routing

Dr. Md. Imdadul Islam

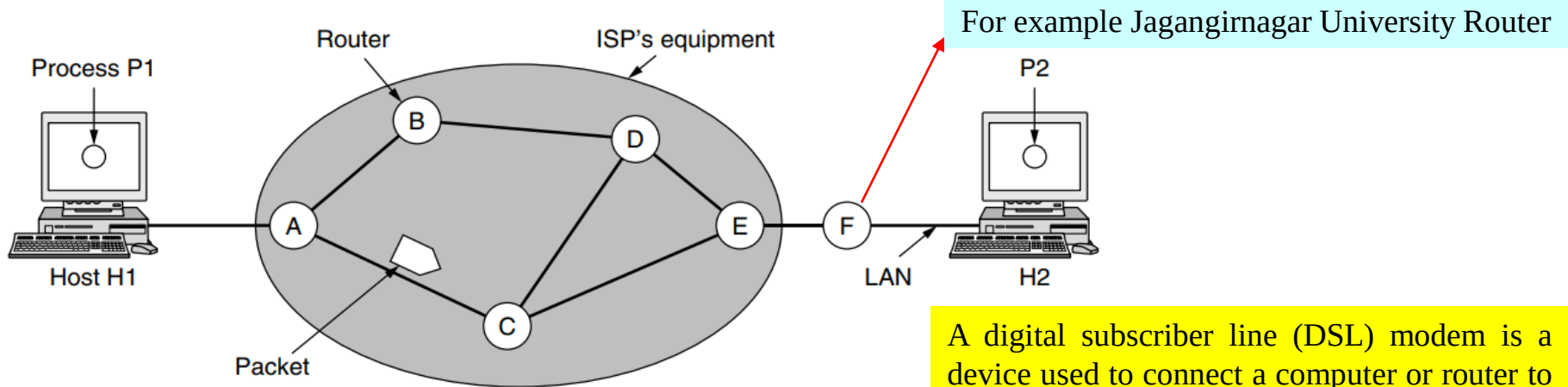
Professor, Department of Computer Science and
Engineering

Jahangirnagar University

<https://www.juniv.edu/teachers/imdad>

Store-and-Forward Packet Switching

The major components of the network are the **ISP's equipment** or **carrier equipment** (routers connected by transmission lines), shown inside the shaded oval, and the customers' equipment, shown outside the oval. Host H1 is directly connected to one of the ISP's routers, A, perhaps as a home computer that is plugged into a DSL modem.



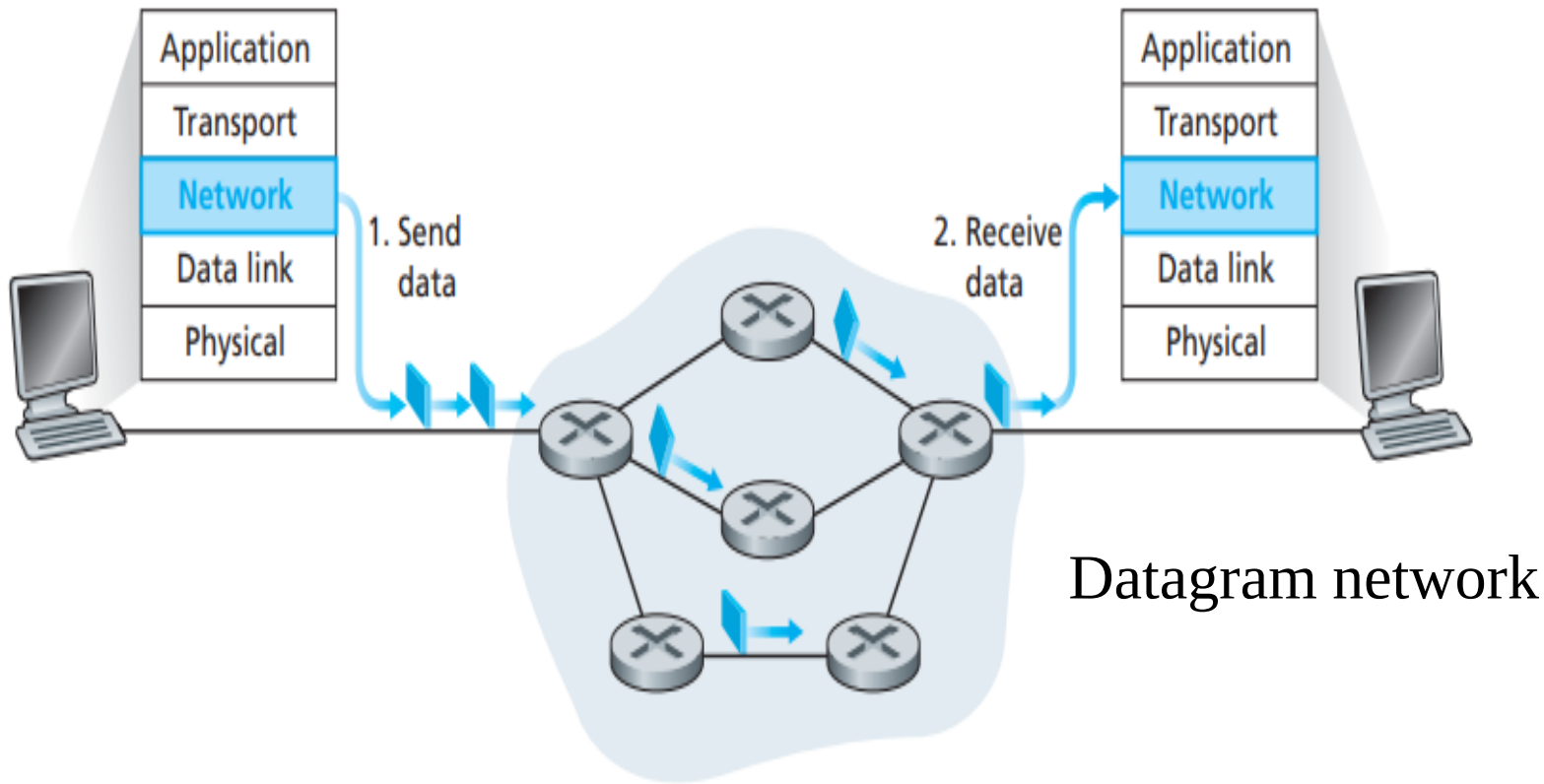
The environment of the network layer protocols.

A digital subscriber line (DSL) modem is a device used to connect a computer or router to a telephone line which provides connection to Internet, which is often called DSL broadband.

In contrast, H2 is on a LAN, which might be an office Ethernet, **with a router, F, owned and operated by the customer**. This router has a leased line to the ISP's equipment. We have shown F as being outside the oval because it does not belong to the ISP.

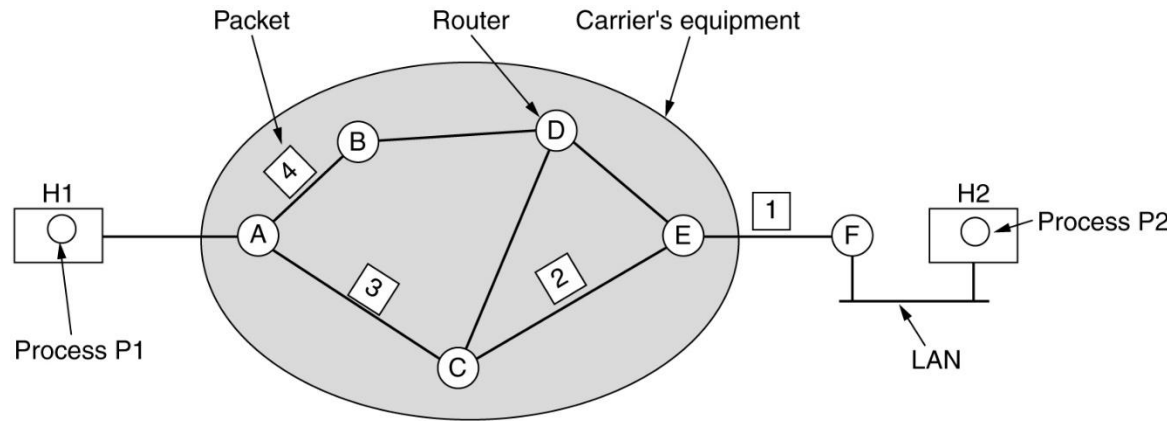
Implementation of Connectionless Service

- ✓ If **connectionless service** is offered, packets are injected onto the subnet individually and routed independently of each other. No advanced setup is needed. In this context, the packets are frequently called **datagram's** (in analogy with telegram) and the network is called a datagram network.
- ✓ If **connection oriented service** is used, a path from the source to destination router must be established before any data packets can be sent. This connection is called a VC (**virtual circuit**), in analogy with the physical circuits set up by the telephone system, and the network is called a virtual circuit network.



In a **datagram network** (connectionless service), each time an **end system** (source) wants to send a packet, it stamps the packet with the **address of the destination end system** (Receiver) and then sends the packet into the network.

Suppose that the process P1 in fig. has a long message for P2. It hand the message to the transport layer with instructions to deliver it to process P2 on host H2. The transport layer code runs on H1, typically within the operating system. It prepends (add something) a transport header to the front of the message and hands the result to the network layer, probably just another procedure within the operating system.



Connectionless Service

Routing tables

S

A's table

initially

later

C's table

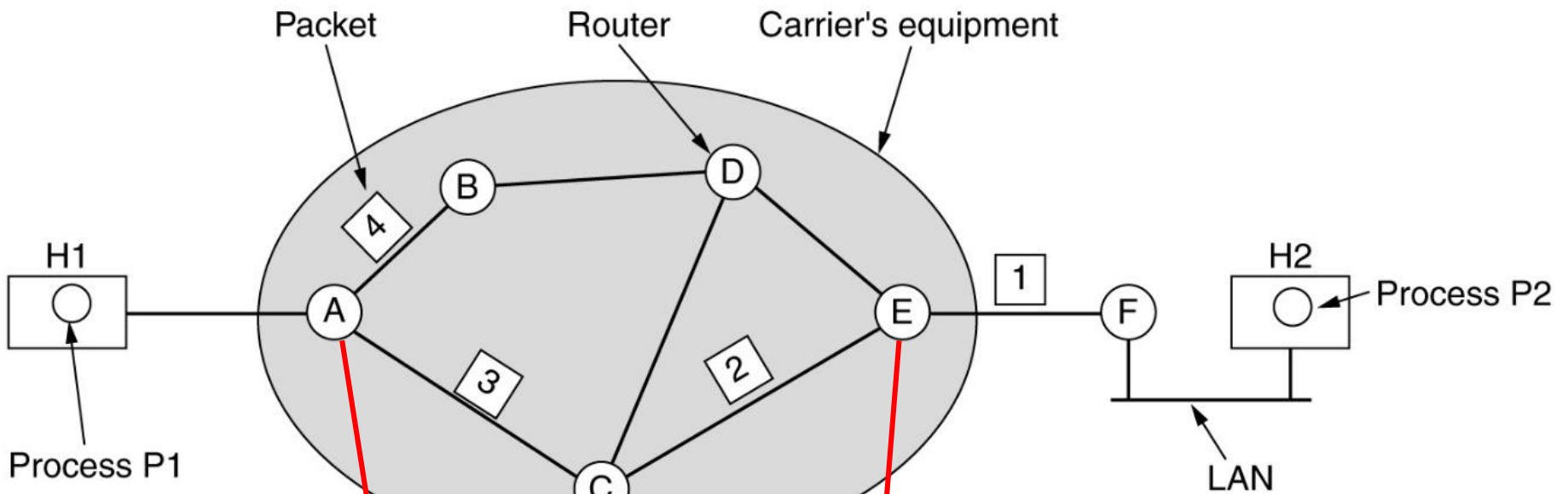
E's table

A	-	A	-	A	A	A	C
B	B	B	B	B	A	B	D
C	C	C	C	C	-	C	C
D	B	D	B	D	D	D	D
E	C	E	B	E	E	E	-
F	C	F	B	F	E	F	F

Dest. Line

The message is four times longer than the maximum packet size, so the transport layer has to break the it into four packets 1, 2, 3 and 4 and deliver to network layer. The network layer sends each of them in turn to router 'A' using some point-to-point protocol.

Routing of datagram in a network



A's table

initially	later
A -	A -
B B	B B
C C	C C
D B	D B
E C	E B
F C	F B

Dest. Line

C's table

A A
B A
C -
D D
E E
F E

E's table

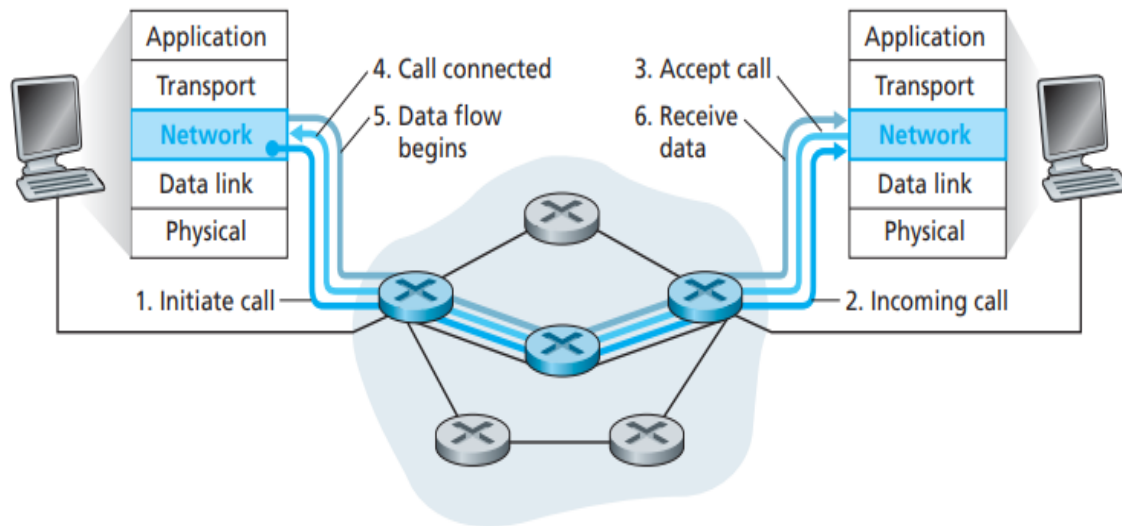
A C
B D
C C
D D
E -
F F

Destination

Via which node

Implementation of Connection-Oriented Service

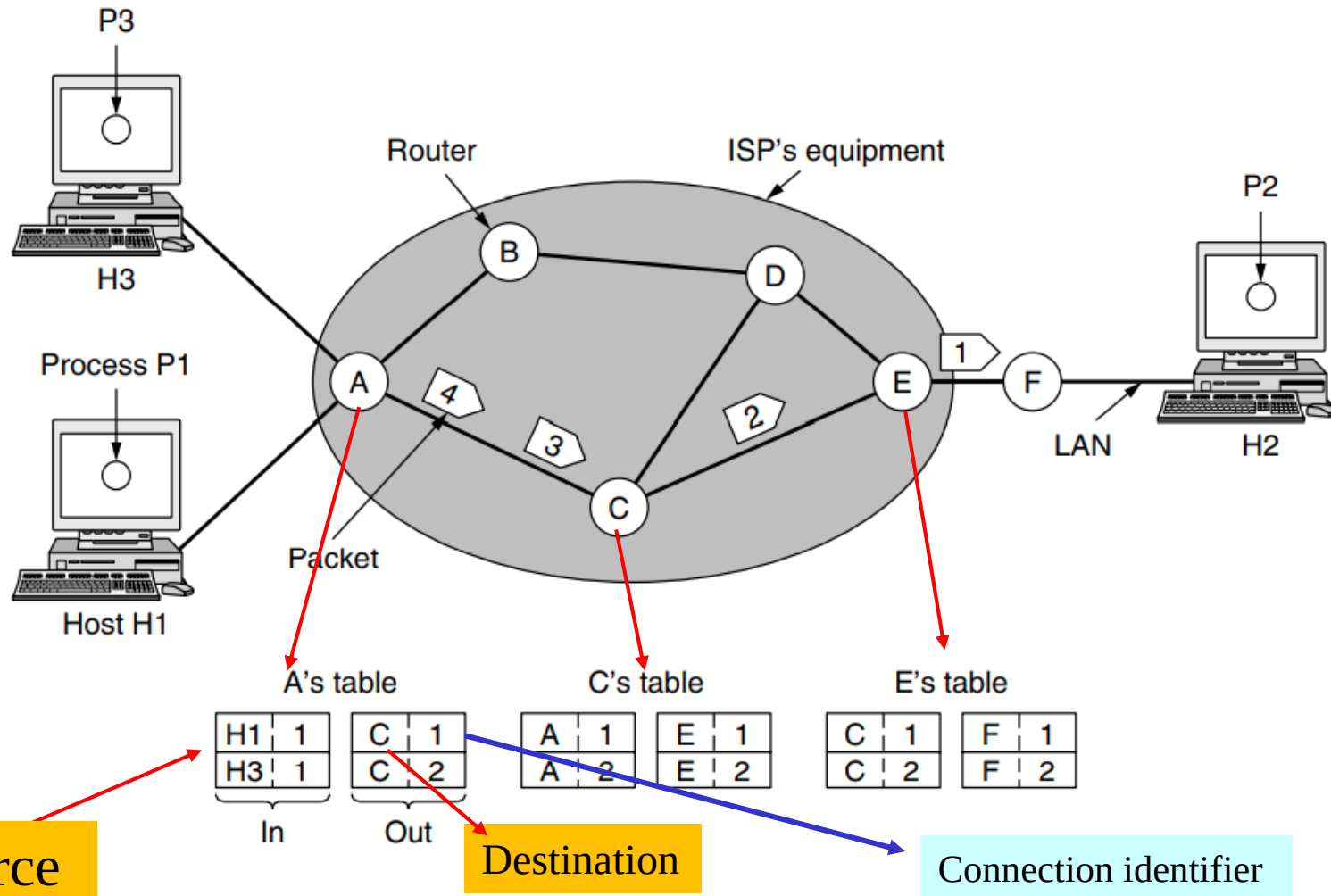
- ✓ A **fixed route** is used for all traffic flowing over the connection, exactly the same way that the telephone system works. When the connection is released, the virtual circuit is also terminated.
- ✓ With connection-oriented service, each packet carries an identifier telling which virtual circuit it belongs to.



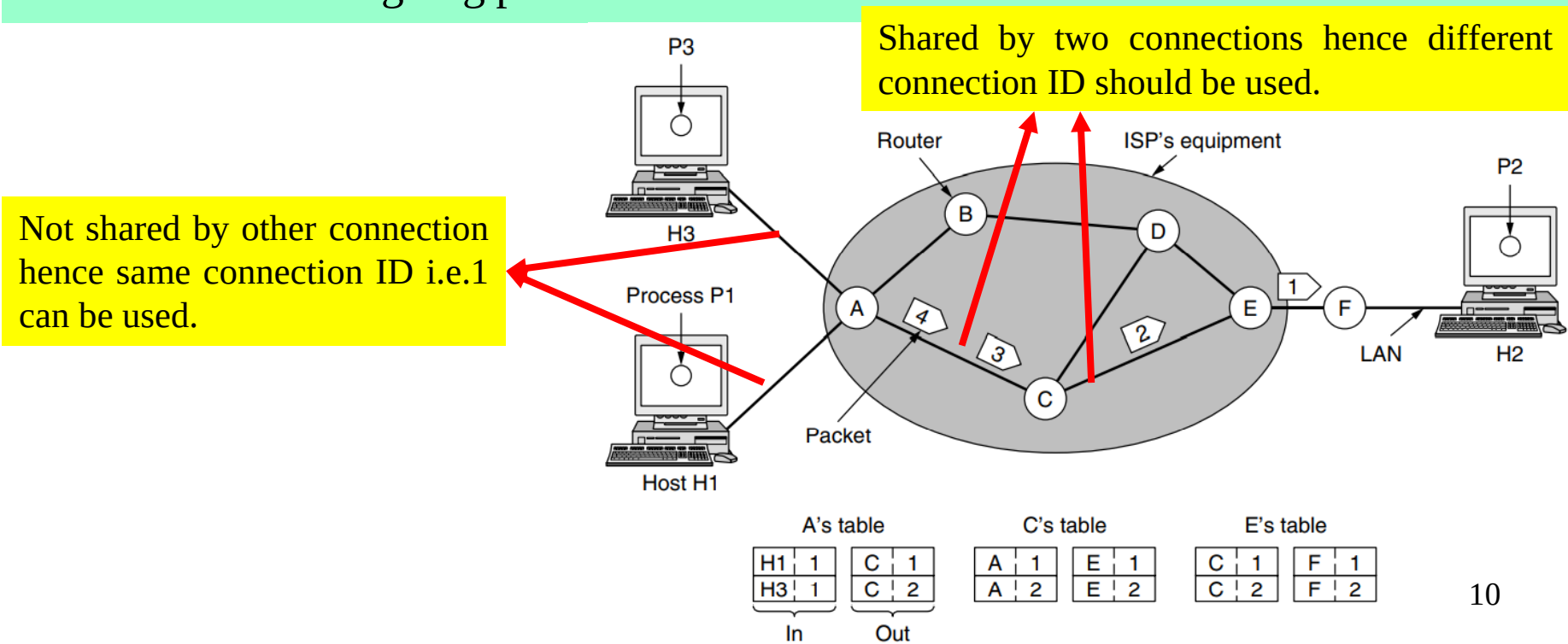
Virtual-circuit setup

Implementation of connection-oriented service

- ✓ Here, host H1 has established connection with host H2. Host H3 also makes connection with host H1.
- ✓ The first line of A's table says that if a packet bearing **connection identifier 1** comes in from H1, it is to be sent to router C and given **connection identifier 2**. Similarly, the first entry at C routes the packet to E, also with **connection identifier 1**.



- ✓ Now let us consider what happens if H3 also wants to establish a connection to H2. It chooses **connection identifier 1** (because it is initiating the connection, and this is its only connection between H3 and A) and tells the network to establish the virtual circuit. This leads to the second row in the tables.
- ✓ Note that we have a conflict here because although A can easily distinguish connection 1 packets from H1 from connection 1 packets from H3, C cannot do this. For this reason, A assigns a different connection identifier to the outgoing traffic for the second connection.
- ✓ Avoiding conflicts of this kind is why routers need the ability to replace connection identifiers in outgoing packets.



Comparison of Virtual-Circuit and Datagram Subnets

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

Routing Algorithms

- The Optimality Principle
- Shortest Path Routing
- Flooding
- Distance Vector Routing
- Link State Routing
- Hierarchical Routing
- Broadcast Routing
- Multicast Routing
- Routing for Mobile Hosts
- Routing in Ad Hoc Networks

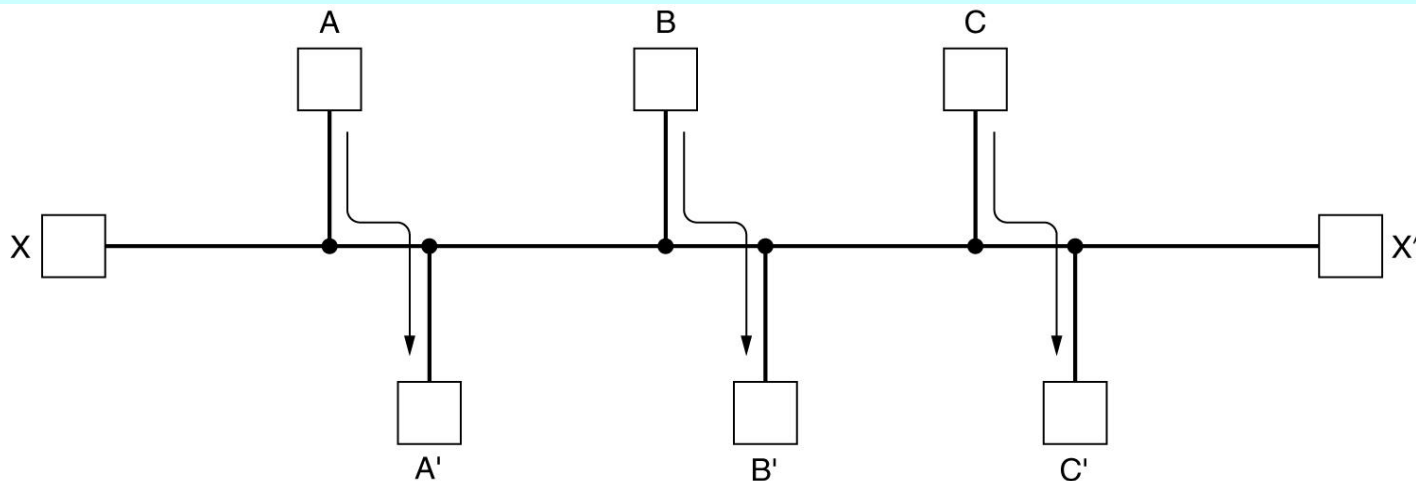
The **routing algorithm** is (part of the network layer software) responsible for deciding which output line an incoming packet should be transmitted on. Certain properties are desirable in a routing algorithm: **correctness**, **simplicity**, **robustness**, **stability**, **fairness** and **optimality**.

Nonadaptive algorithms (static routing) do not change their routing decisions on any measurements or estimates of the current **topology** and **traffic**.

Adaptive algorithms, in contrast, change their routing decisions to reflect changes in the **topology**, and sometimes changes in the **traffic** as well.

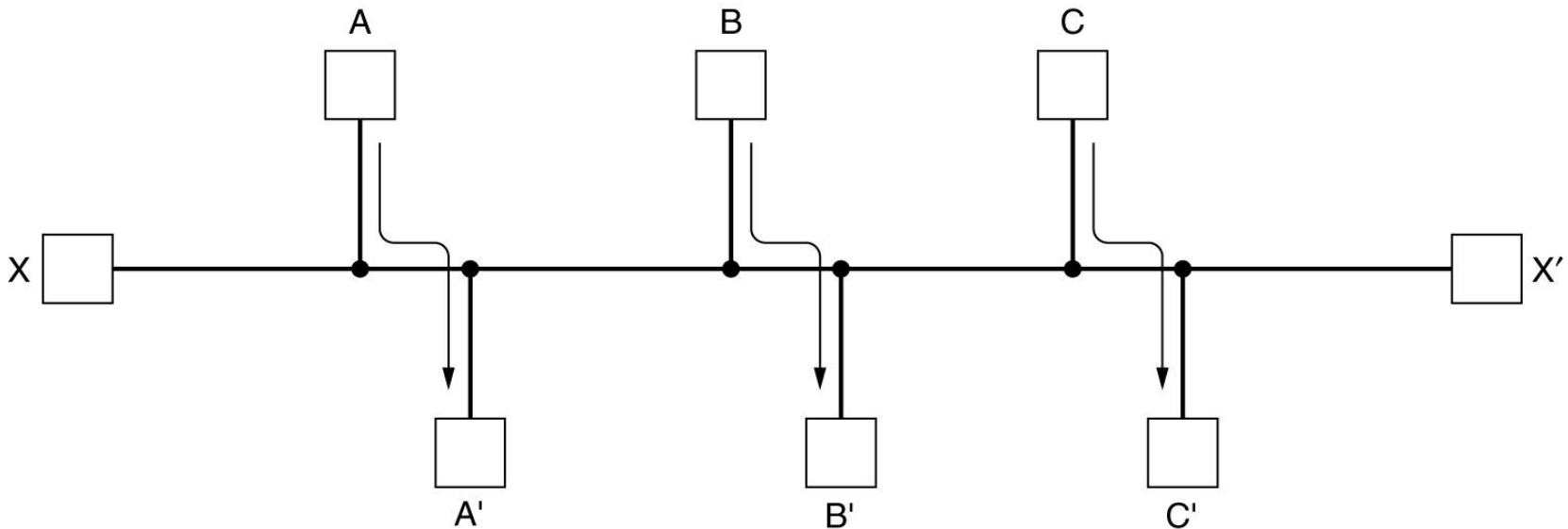
Conflict between fairness and optimality in packet routing

- ✓ **Fairness** and **optimally** should be the goal of any engineering problem but sometimes they bring contradictory goals.
- ✓ **Fairness** means equal opportunity to all connected nodes.
- ✓ In context of **optimality**, we actually need to reduce delay of packet. The routing algorithm attempt to minimize the number of hops to reduce delay and increase the throughput i.e. maximize the **efficiency** of the network.



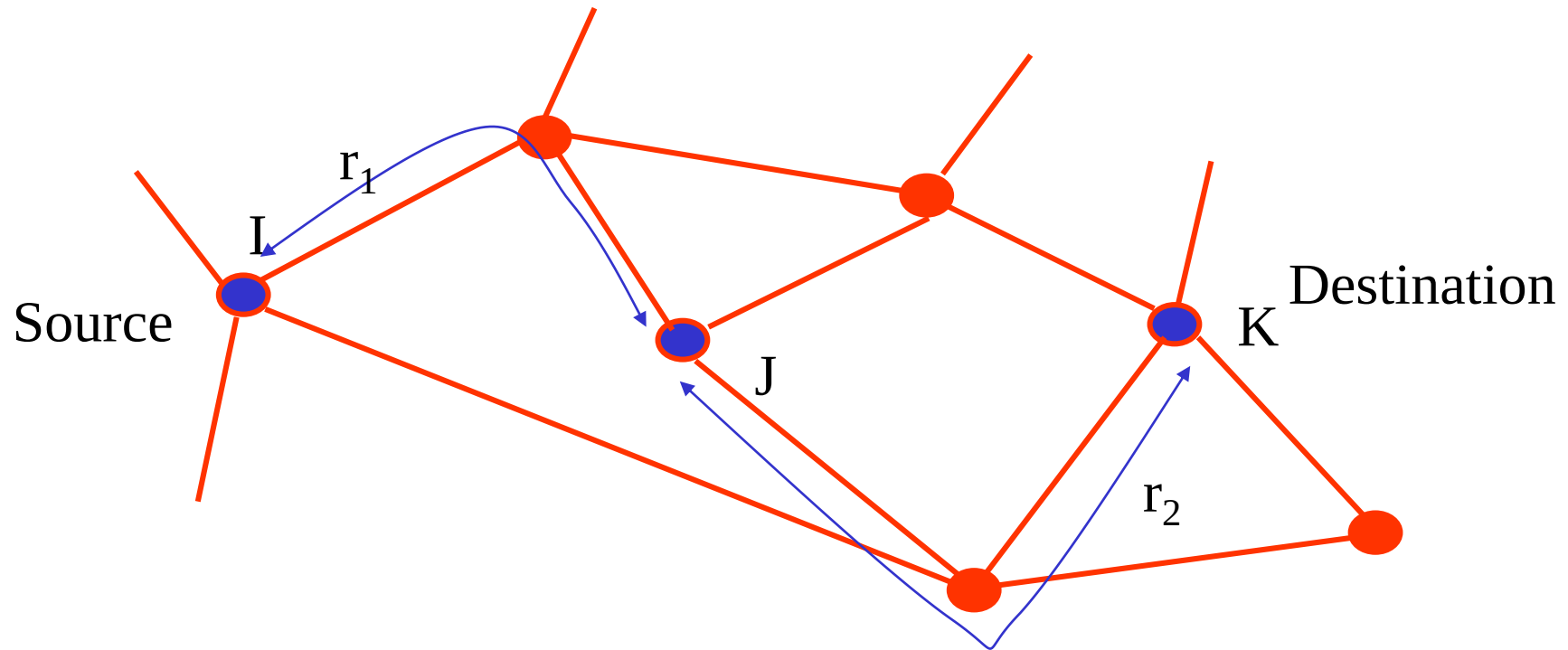
Conflict with efficiency and fairness:

Suppose that there is enough traffic between A and A', between B and B', and between C and C' to saturate the horizontal links. To maximize the total flow, the X and X' traffic should be shut off altogether. Unfortunately, X and X' may not see it that way. Evidently **some compromise** between global **efficiency** and **fairness** to individual connections is needed.



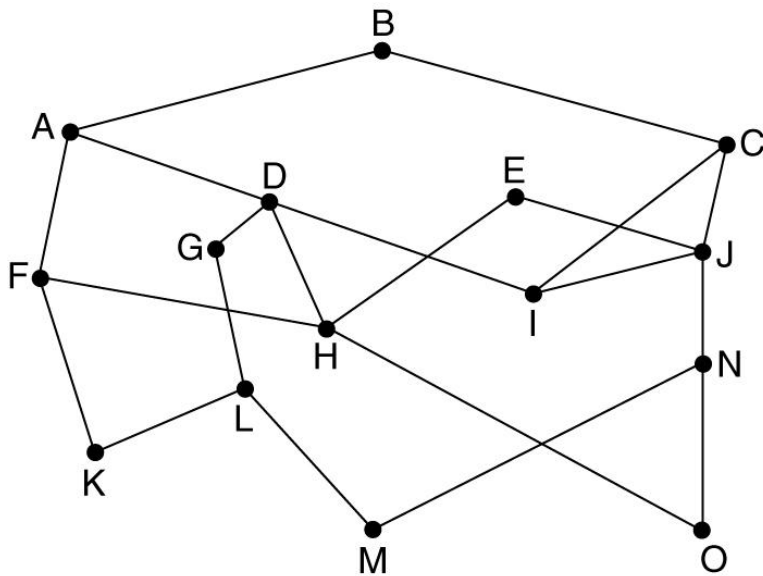
The Optimality Principle

- ✓ It states that **if router J is on the optimal path** from router I to router K, then the optimal path from J to K also falls along the same route.
- ✓ To see this, call the part of the route from I to J is r_1 and the rest of the route r_2 . If a route better than r_2 existed from J to K, it could be concatenated with r_1 to improve the route from I to K, contradicting our statement that r_1+r_2 is optimal.

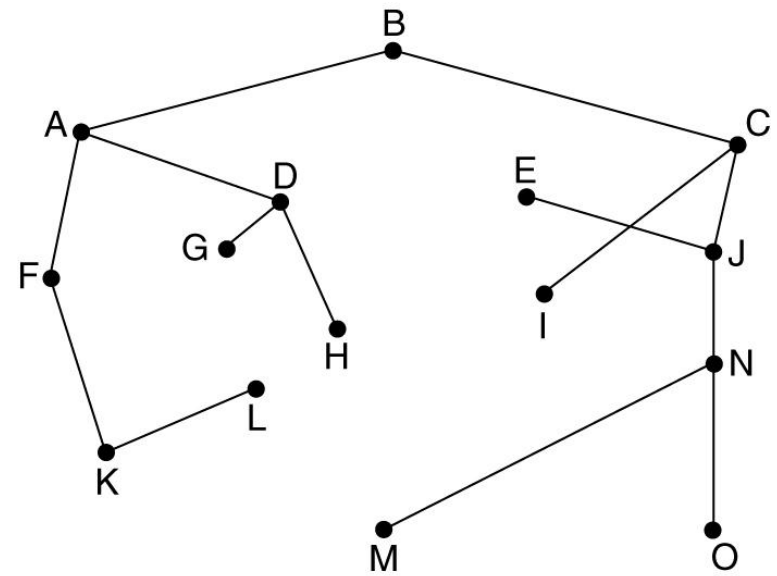


✓ A subnet is represented by a graph, $G = (V, E)$, where routers are the vertices and links are the edges. A tree is a connected undirected graph with no simple circuits

✓ Consider a terminal of a network as the destination and the rest of the node are sources. Draw the optimal path from each source to the destination node. You will get a rooted tree where the destination node will be the root of the tree. Such tree is called sink tree. The sink tree of the network is not unique.



(a)



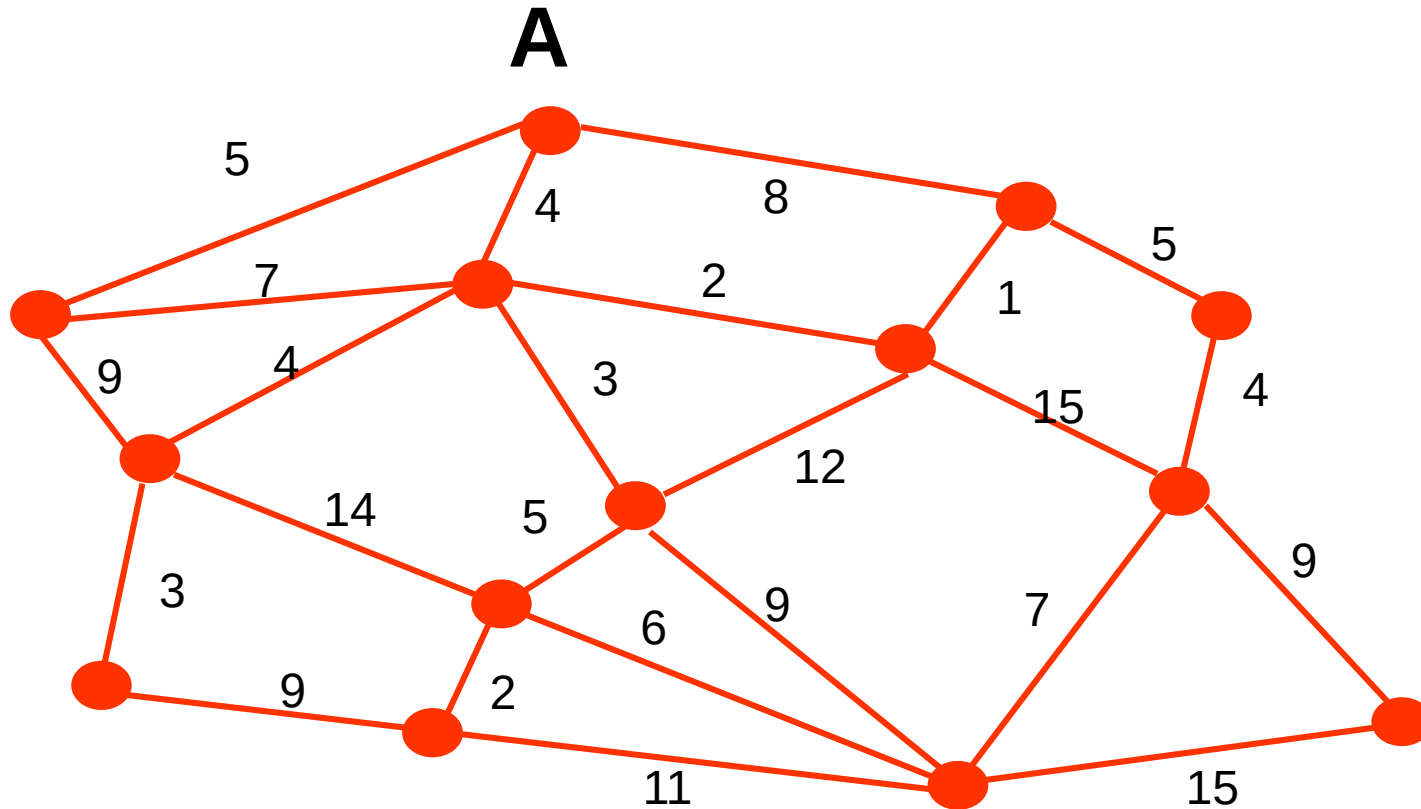
(b)

(a) A subnet. (b) A sink tree for router B

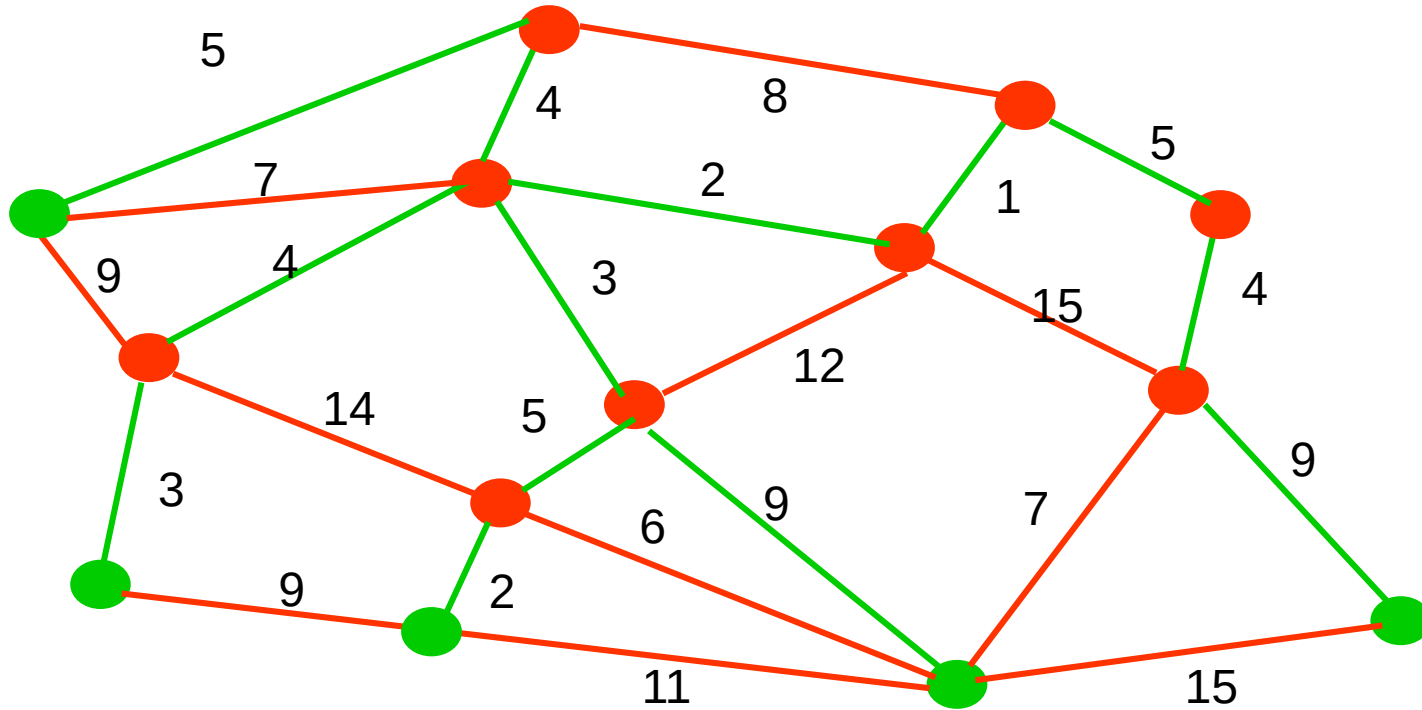
What is the drawback of such algorithm?

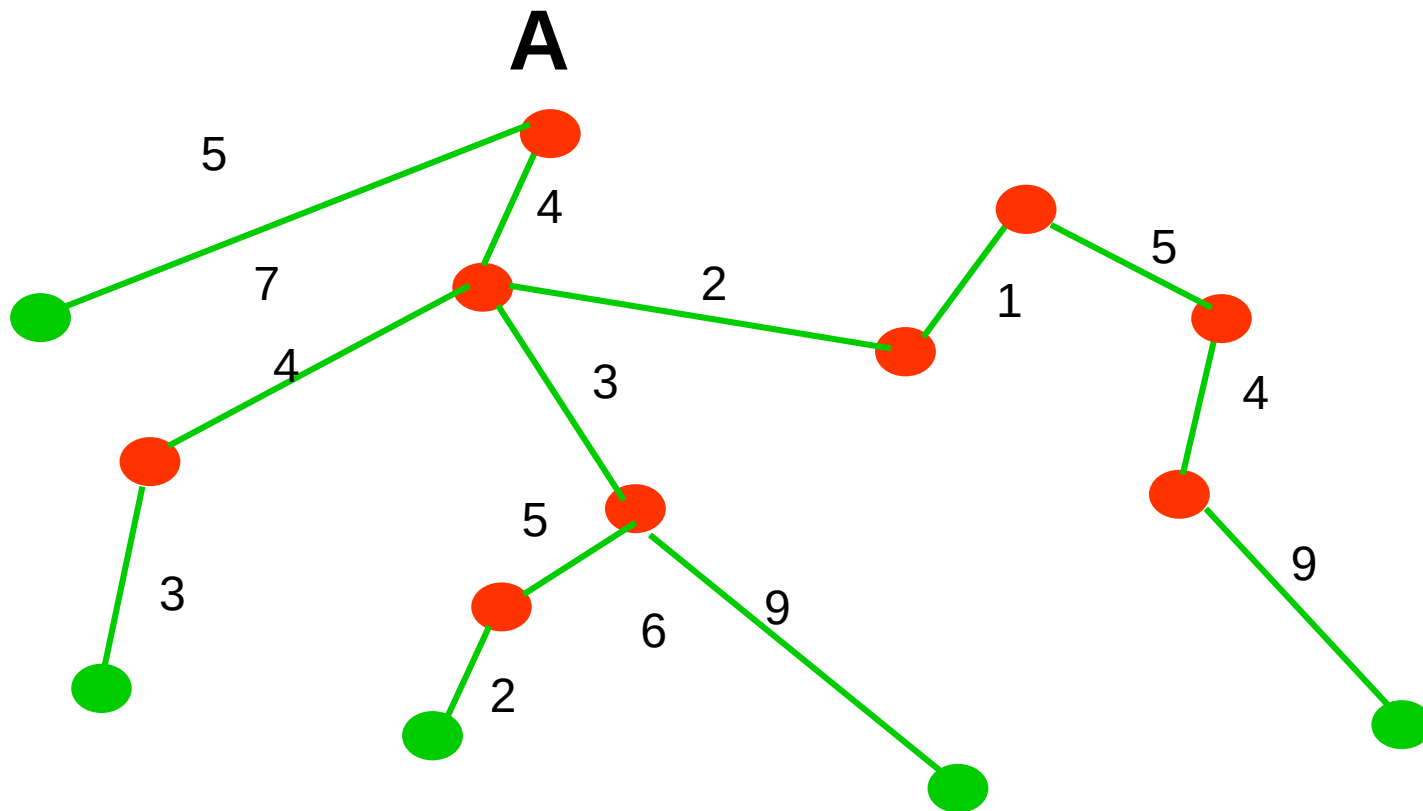
Exercise

Draw the sink tree taking A as the root/destination.



A





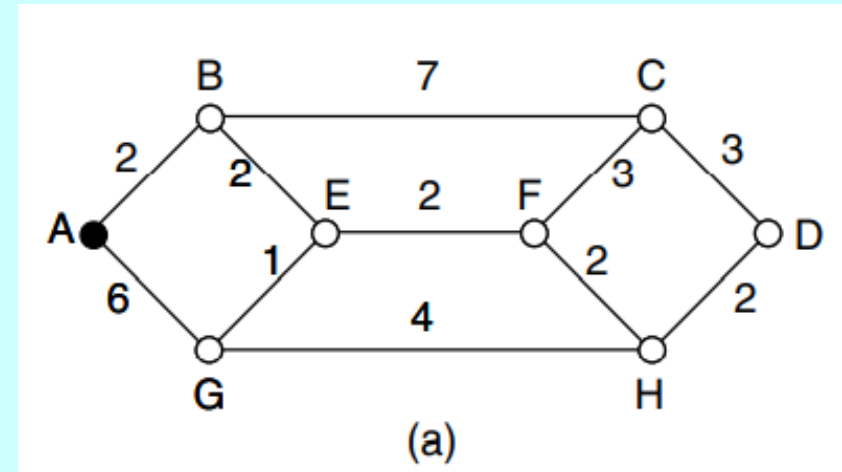
Shortest Path Routing

✓ The concept of shortest path deserves some explanation. One way of measuring path length is the *number of hops*. Using this metric, path $ABC = \text{path } ABE$.

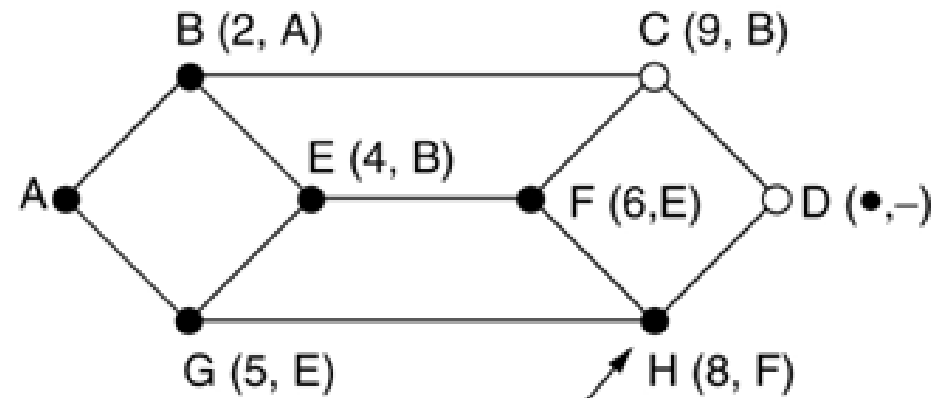
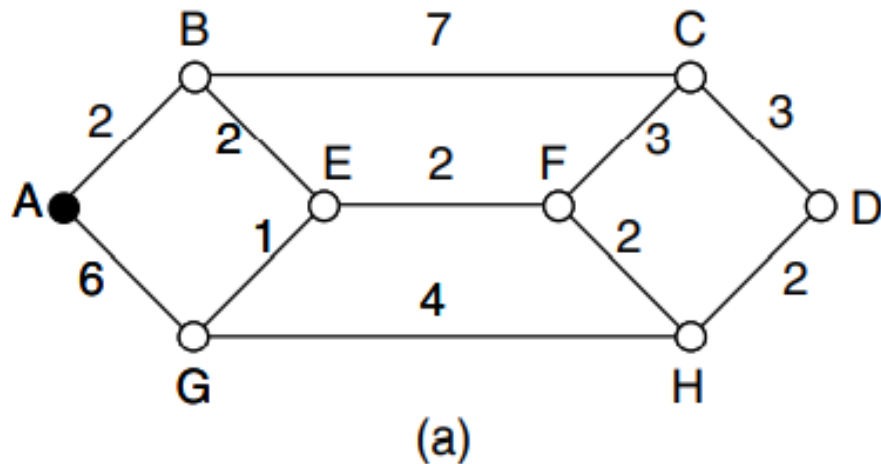
✓ Another metric is the *geographic distance in Km*, in this case path $ABC > \text{path } ABE$.

Usually path distance is function of:

- ❖ propagation delay,
- ❖ BW/ link capacity in bps,
- ❖ average traffic load on the link,
- ❖ communication cost (in case of leased line),
- ❖ mean queue length of routers along the path (related to delay) etc.
- ❖ The combinations of above criteria the algorithm measures the shortest path.

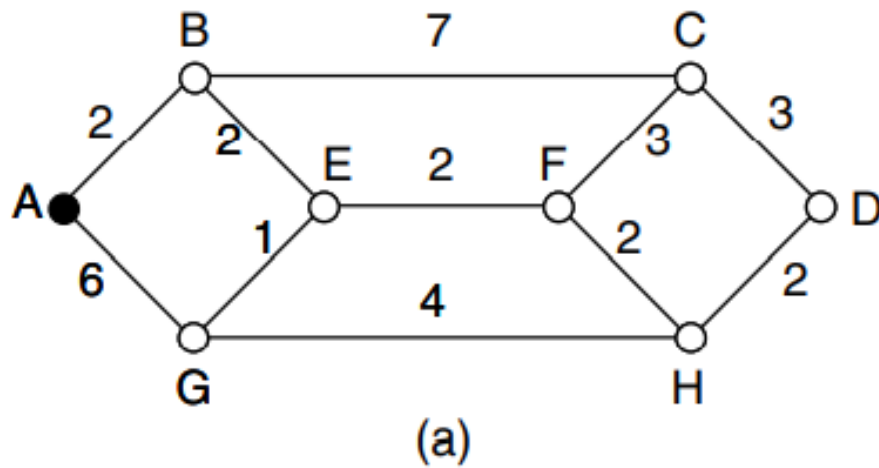


- ✓ Each link is weighted using above criteria (weighted graph). Each node is labeled (in parentheses) with its distance from the source node.
- ✓ Initially, no paths are known, so all nodes are labeled with infinity.
- ✓ As the algorithm proceeds and path are found, the labels may change, reflecting better paths.



Source node is E and distance is 5

Dijkstras Shortest Path Algorithm

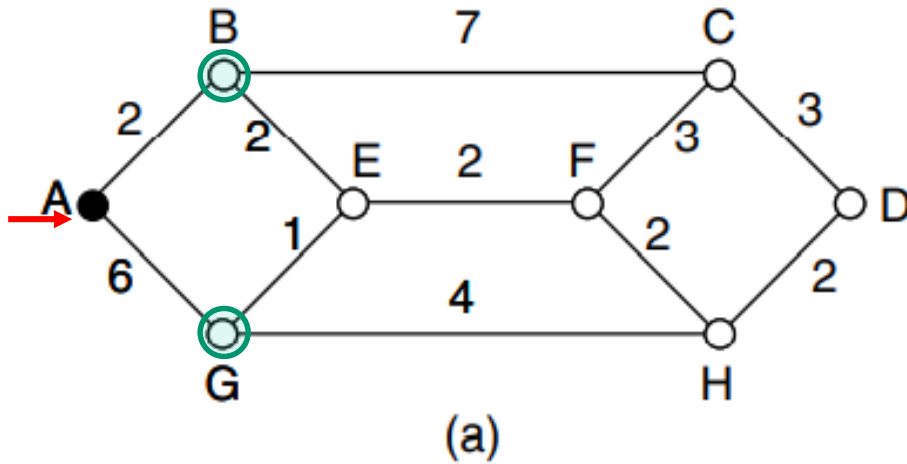


Consider **node-A** as the source and the OSPF will determine the shortest path from node A. Initially weights are kept infinity.

Destination node	Weight	Previous node
A	∞	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	
G	∞	
H	∞	

Remaining (unvisited) nodes:
 {A, B, C, D, E, F, G, H}

Visited nodes: { }



Consider node-A as the source and the OSPF will determine the shortest path from node A.

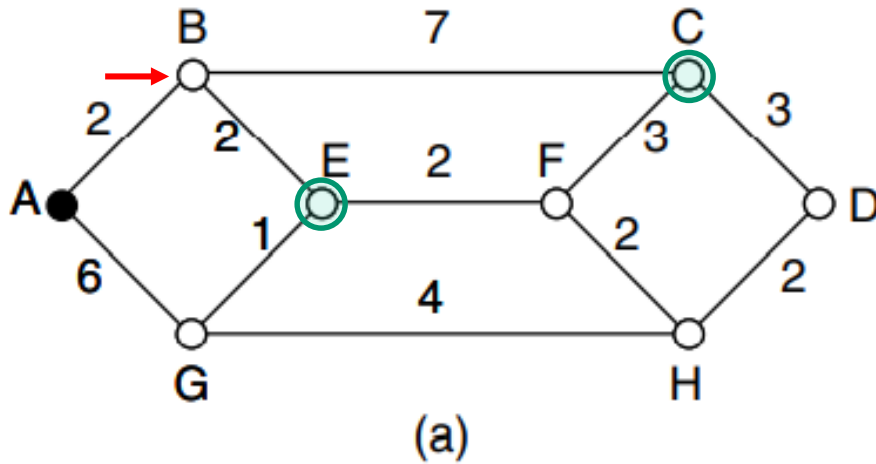
Destination node	Weight	Previous node
A	0	
B	2 (min)	A
C	∞	
D	∞	
E	∞	
F	∞	
G	6	A
H	∞	

Remaining nodes:
{B, C, D, E, F, G, H}

Visited nodes: {A}

Node-A has two adjacent nodes B and G. Update their weights and marks A as their previous node.

Next, choose the node with minimum weight i.e. node B.

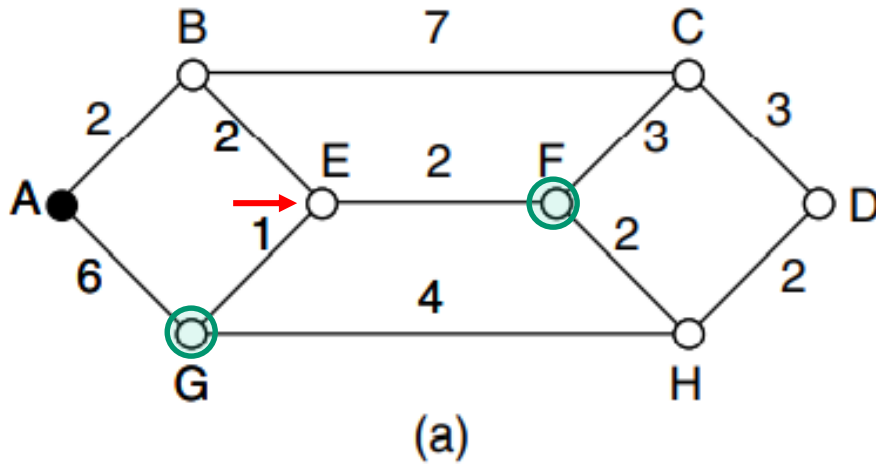


Consider node-A as the source and the OSPF will determine the shortest path from node A.

Destination node	Weight	Previous node
A	0	
B	2	A
C	$2 + 7 = 9$	B
D	∞	
E	$2 + 2 = 4$ (min)	B
F	∞	
G	6	A
H	∞	

Remaining nodes:
{C, D, E, F, G, H}

Visited nodes:
{A, **B**}

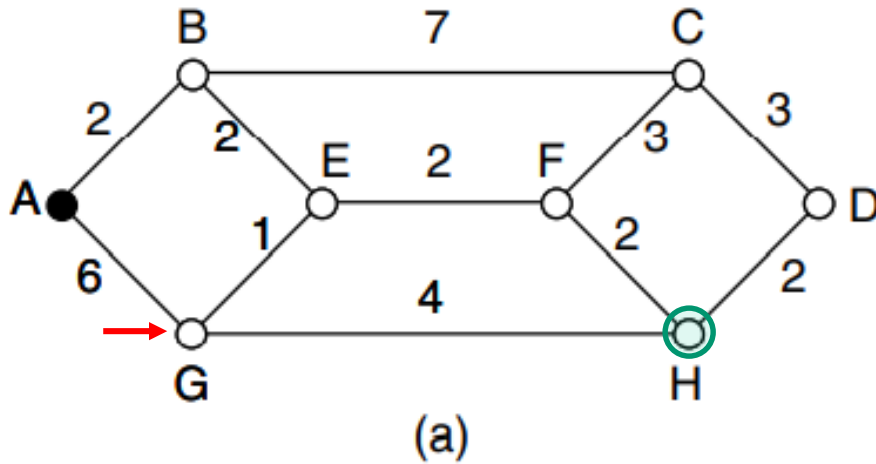


Consider node-A as the source and the OSPF will determine the shortest path from node A.

Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	∞	
E	4	B
F	$4 + 2 = 6$	E
G	$4 + 1 = 5$ (min)	E
H	∞	

Remaining nodes:
{C, D, F, G, H}

Visited nodes:
{A, B, **E**}

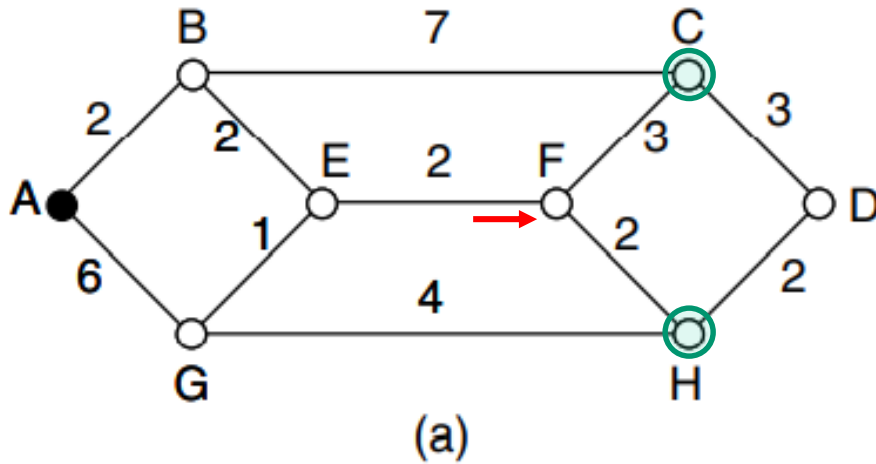


Consider node-A as the source and the OSPF will determine the shortest path from node A.

Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	∞	
E	4	B
F	6	E
G	5	E
H	$5 + 4 = 9$	G

Remaining nodes:
{C, D, F, H}

Visited nodes:
{A, B, E, **G**}



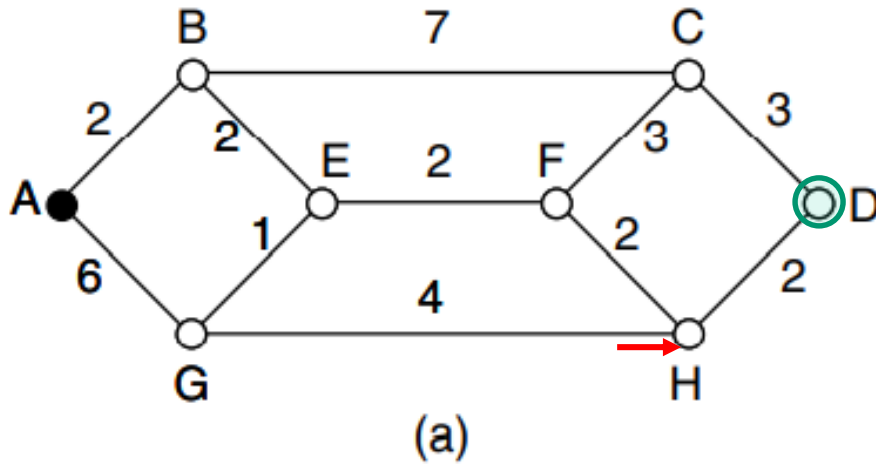
Consider node-A as the source and the OSPF will determine the shortest path from node A.

Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	∞	
E	4	B
F	6	E
G	5	E
H	$6 + 2 = 8$	F

Remaining nodes:
{C, D, H}

Visited nodes:
{A, B, E, G, **F**}

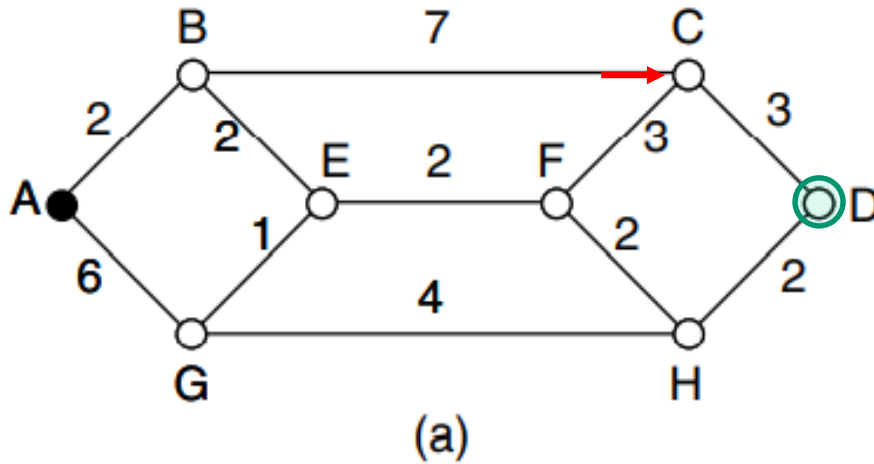
F cannot improve the weight of C so weights are kept like before and its previous node remains B.



Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	$8 + 2 = 10$	H
E	4	B
F	6	E
G	5	E
H	8	F

Remaining nodes: {C, D}

Visited nodes: {A, B, E, G, F, **H**}



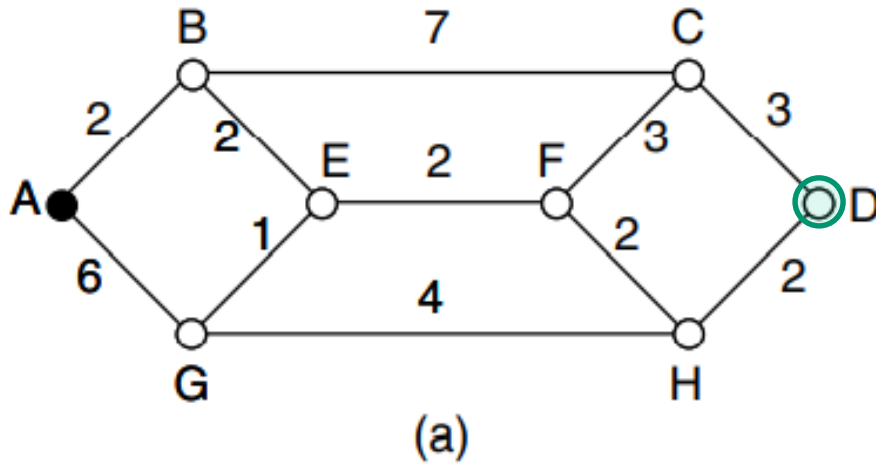
Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	10	H
E	4	B
F	6	E
G	5	E
H	8	F

Remaining nodes: {D}

Visited nodes: {A, B, E, G, F, H, C}

Node C can not improve the weights of D so the previous node of D remains H like before.

Now the only unvisited node is D and it has no unvisited adjacent node, therefore algorithm will be terminated.



Destination node	Weight	Previous node
A	0	
B	2	A
C	9	B
D	10	H
E	4	B
F	6	E
G	5	E
H	8	F

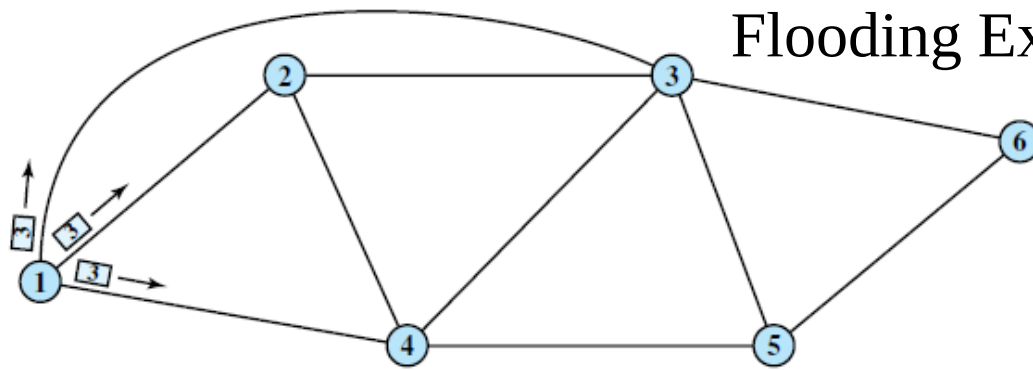
From the table the shortest path from A to D is,
D-H (4th row) + H-F(8th row) + F-E(6th row) + E-B(5th row)
+ B-A (2nd row)
= D-H-F-E-B-A

Flooding

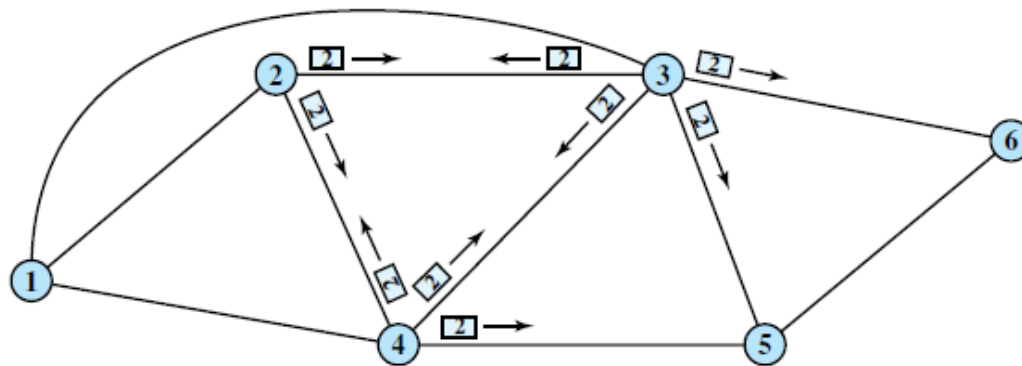
Flooding is a static algorithm in which every incoming packet is sent out on every outgoing line except the one it arrived on like HUB. Flooding obviously generates vast numbers of duplicating packets. Several **remedial measures** are taken against huge duplications of packets:

- Using hop counter in the header of each packet.
- An alternative technique for damming the flood is to keep track of which packets have been flooded, to avoid sending them out a second time.
- Using selective flooding algorithm where the routers do not send every incoming packet out on every line, only on those lines that are going approximately in the right direction.

Flooding Example: hop count = 3

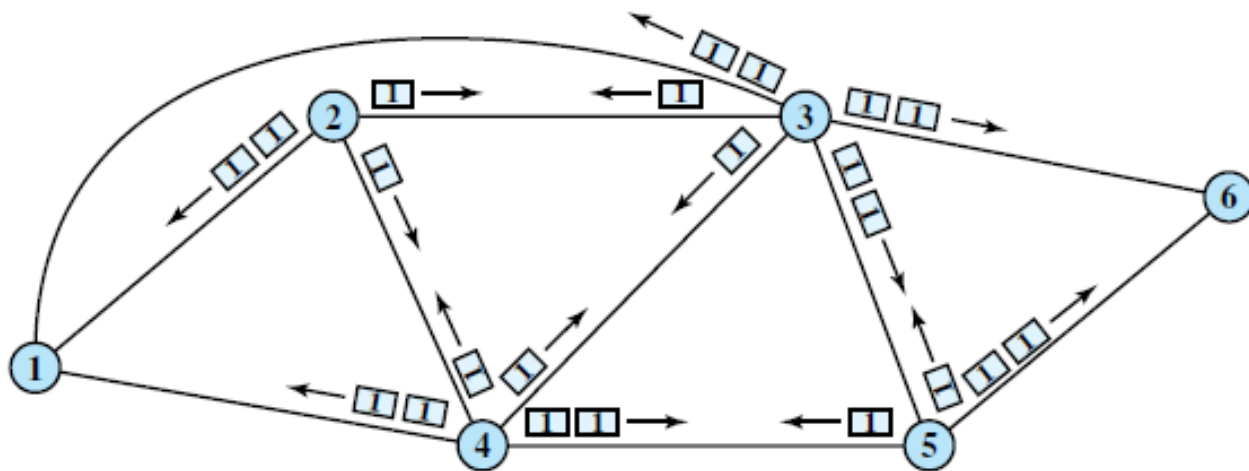


(a) First hop



(b) Second hop

hop count = 2



(c) Third hop

hop count = 1

- Flooding is not practical in most applications, but it does have some uses. For example, in military applications, where a large number of routers may be blown at any instant, the tremendous robustness of flooding is highly desirable.
- In wireless networks, all messages are transmitted by a station can be received by all other stations within its radio range (because of unguided communications) which is in fact flooding.